

Implementation of Processes

Definition

To implement the process model, the operating system keeps a **process table**, usually with one entry for each process. Each entry stores everything needed to stop a process, keep track of it, and later restart it correctly.

1. The Process Table

Main Idea

The **process table** is an array of data structures maintained by the operating system. Each entry describes one process.

Another Name

Many authors call a process-table entry a **process control block (PCB)**.

Why It Is Needed

Whenever a process changes from **running** to **ready** or **blocked**, the operating system must save enough information so the process can later resume as if it had never been interrupted.

2. What the Process Table Stores

Typical Information

A process-table entry usually contains:

- CPU-related state such as the program counter and registers,
- stack information,
- memory-management information,
- open-file information,
- scheduling data,
- accounting data,
- process identity and relationship information.

Process management	Memory management	File management
Registers, program counter, program status word, stack pointer, process state, priority, scheduling parameters, process ID, parent process, process group, signals, start time, CPU time used, children's CPU time, next alarm time	Pointer to text segment information, pointer to data segment information, pointer to stack segment information	Root directory, working directory, file descriptors, user ID, group ID

System Dependent

The exact fields in a process table vary from one operating system to another. The important idea is not the exact list, but that the OS must store all information needed to suspend and resume a process correctly.

3. How One CPU Appears to Run Many Processes

Core Illusion

Even on a single CPU, the operating system creates the illusion of multiple sequential processes by rapidly saving the state of one process and restoring the state of another.

What Must Be Preserved

For this illusion to work, the OS must save the interrupted process state precisely and later restore it precisely.

4. Interrupt Vectors

Interrupt Vector

For each I/O class, the system keeps an **interrupt vector**, usually at a fixed location in memory. It stores the address of the interrupt service procedure that should run when that interrupt occurs.

Example Situation

Suppose **user process 3** is running when a disk interrupt occurs. The interrupt hardware immediately saves a small amount of execution state and transfers control to the interrupt service routine.

5. What the Hardware Does First

Initial Hardware Action

When an interrupt happens, the hardware typically:

- pushes the current program counter onto the current stack,
- pushes the program status word,
- sometimes pushes one or more registers,
- then jumps to the address stored in the interrupt vector.

Key Limit

That is usually all the hardware does. After that, the rest of the work is handled by software, especially the interrupt service procedure.

6. What the Software Does Next

Saving the Current Process

All interrupts begin by saving the processor registers, often into the process-table entry of the currently running process.

Temporary Stack

After that, the information pushed by the hardware is removed from the old stack, and the stack pointer is changed to a temporary stack used by the interrupt handler.

Why Assembly Is Needed

Low-level actions such as saving registers and changing the stack pointer cannot be expressed directly in high-level languages like C, so a short **assembly-language routine** performs them first.

Same for All Interrupts

This low-level assembly routine is often shared by all interrupts, because saving registers is basically the same operation no matter what caused the interrupt.

7. Role of the C Interrupt Handler

Higher-Level Processing

Once the low-level assembly work is finished, it calls a **C procedure** that handles the specific interrupt type.

What This Handler May Do

The C interrupt handler may:

- process the interrupt event,
- update operating-system data structures,
- mark some blocked process as ready,
- request that the scheduler choose the next process to run.

8. Scheduler and Process Restart

Scheduler Step

After the interrupt has been handled, the **scheduler** is called to decide which process should run next.

Restoring the Next Process

Control then returns to low-level code, which:

- loads the registers of the selected process,
- restores its memory map,
- and starts that process running.

Essential Result

If the same process is resumed later, it continues from exactly the same state it had before the interrupt.

9. Interrupt Handling Flow

Typical Sequence

The overall sequence is:

1. A process is running.
2. An interrupt occurs.
3. Hardware saves minimal state and jumps through the interrupt vector.
4. Assembly code saves registers and switches to a temporary stack.
5. A C interrupt handler processes the event.
6. The scheduler decides which process should run next.
7. Low-level code restores that process's registers and memory map.
8. Execution resumes.

10. Why This Works

Key Insight

A process may be interrupted thousands of times while it runs, but after each interrupt it can resume as though nothing happened, because its full execution context was preserved.

Big Picture

This save-and-restore mechanism is what makes process switching, scheduling, and the whole process model work in practice.

11. Quick Review

Exam-Focused Points

- The operating system implements processes using a **process table**.
- Each process-table entry stores everything needed to suspend and resume a process.
- A process-table entry is often called a **process control block (PCB)**.
- The exact table fields are system dependent, but usually cover process, memory, and file management.
- Interrupt vectors store the addresses of interrupt service procedures.
- Hardware saves a small amount of state first, then software does the rest.
- Low-level assembly code saves registers and switches stacks.
- A C interrupt handler processes the interrupt and may wake blocked processes.
- The scheduler selects the next process, and its state is restored.
- A resumed process continues from the same state it had before the interrupt.